

GLOBALRAIN

Practices for Secure Software Report

Table of Contents

DOCUMENT REVISION HISTORY 3
CLIENT 3
INSTRUCTIONS 3
DEVELOPER 4
1. ALGORITHM CIPHER 4
2. CERTIFICATE GENERATION 4
3. DEPLOY CIPHER 4
4. SECURE COMMUNICATIONS 4
5. SECONDARY TESTING 4
6. FUNCTIONAL TESTING 4
7. SUMMARY 4
8. INDUSTRY STANDARD BEST PRACTICES 4

Document Revision History

Version	Date	Author	Comments
1.0	10/23/2026	Louis Richards	

Client



Instructions

Submit this completed practices for secure software report. Replace the bracketed text with the relevant information. You must document your process for writing secure communications and refactoring code that complies with software security testing protocols.

- Respond to the steps outlined below and include your findings.
- Respond using your own words. You may also choose to include images or supporting materials. If you include them, make certain to insert them in all the relevant locations in the document.
- Refer to the Project Two Guidelines and Rubric for more detailed instructions about each section of the template.

Developer
Louis Richards

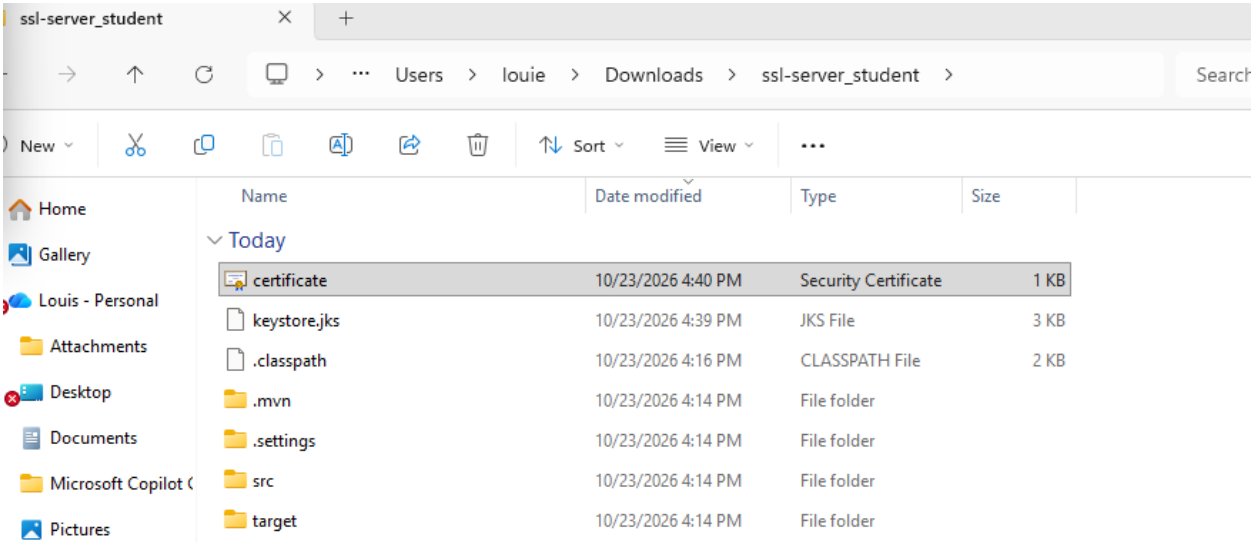
1. Algorithm Cipher

For Artemis Financial, I recommend using the Advanced Encryption Standard (AES) to help protect sensitive financial information. It's a symmetric encryption algorithm, meaning it uses the same key to encrypt and decrypt data. It supports 128, 192, and 256-bit keys, with longer keys providing stronger protection. Its security and efficiency helped it replace older encryption methods like DES, and it's still widely used today.

AES also uses multiple rounds of encryption to make data difficult to read without the correct key. Randomly generated keys help prevent attackers from guessing them, while asymmetric encryption uses separate public and private keys. For this project, I used SHA-256 to generate a checksum. Unlike AES, SHA-256 is a hashing algorithm that creates a 256-bit value from the original data. This value can be compared with another checksum to help identify whether the data has been changed. Together, encryption and hashing can help protect sensitive information and verify its integrity.

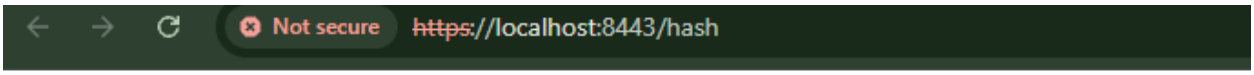
2. Certificate Generation

Insert a screenshot below of the CER file.



3. Deploy Cipher

Insert a screenshot below of the checksum verification.



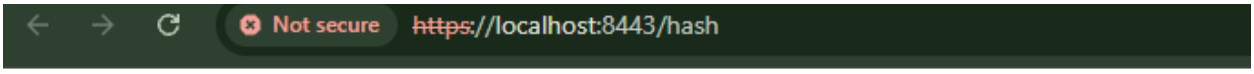
data: Louis Richards

Name of Cipher Algorithm Used: SHA-256

CheckSum Value: 5fcbfb7f4524e8462f43d683655cda27cfe9a09a606e81331d8e6e7ebf97d661

4. Secure Communications

Insert a screenshot below of the web browser that shows a secure webpage.



data: Louis Richards

Name of Cipher Algorithm Used: SHA-256

CheckSum Value: 5fcbfb7f4524e8462f43d683655cda27cfe9a09a606e81331d8e6e7ebf97d661

5. Secondary Testing

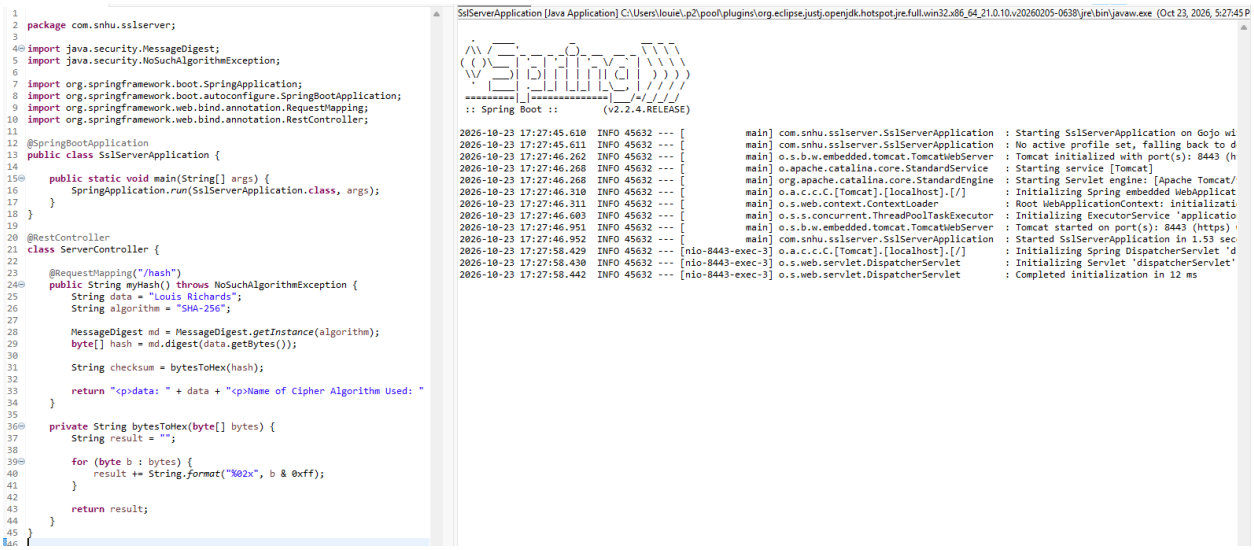
Insert screenshots below of the refactored code executed without errors and the dependency-check report.





6. Functional Testing

Insert a screenshot below of the refactored code executed without errors.



```
package com.snhu.sslserver;

import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@SpringBootApplication
public class SslServerApplication {

    public static void main(String[] args) {
        SpringApplication.run(SslServerApplication.class, args);
    }
}

@RestController
class ServerController {

    @RequestMapping("/hash")
    public String myHash() throws NoSuchAlgorithmException {
        String data = "Louis Richards";
        String algorithm = "SHA-256";

        MessageDigest md = MessageDigest.getInstance(algorithm);
        byte[] hash = md.digest(data.getBytes());

        String checksum = bytesToHex(hash);

        return "<p>data: " + data + "<p>Name of Cipher Algorithm Used: "
    }

    private String bytesToHex(byte[] bytes) {
        String result = "";

        for (byte b : bytes) {
            result += String.format("%02x", b & 0xff);
        }

        return result;
    }
}
```

7. Summary

For Artemis Financial, I made several changes to improve the security of its application. I added SHA-256 hashing to generate a checksum, created a self-signed certificate using Java Keytool, and configured the app to use HTTPS instead of HTTP. These changes help protect data during communication and provide a way to check its integrity.

During the vulnerability assessment process, I focused mainly on cryptography, client-server communication, and code quality. I reviewed the Java code for errors and tested the application to make sure the checksum worked correctly. I also used OWASP Dependency-Check to scan for known vulnerabilities. The scan found several vulnerabilities in older libraries, including Spring Boot, Jackson, and Tomcat. These were part of the original application and still need to be addressed.

By adding these layers of security, the application is better protected than before. HTTPS helps encrypt communication, while SHA-256 provides a way to detect changes to data. Testing the code and reviewing dependencies also help identify security problems before they can cause harm. Although the application still has some vulnerabilities to address, these improvements are a good first step.

8. Industry Standard Best Practices

While updating Artemis Financial's application, I followed several security practices to help protect its data. I used SHA-256 to create a checksum, generated an SSL certificate, and added HTTPS to secure communication between the client and server. I also kept most of the original code and tested my changes to make sure everything still worked. Using OWASP Dependency-Check helped me find vulnerabilities in older libraries that may need to be updated.
Following these practices is important because even small security problems can put customer information at risk. For a financial company like Artemis Financial, a data breach could lead to financial losses and a loss of customer trust. Regular testing, keeping software updated, and using secure connections can help prevent these problems. These steps can help the company protect its customers and keep its application running safely.